

Technische Hochschule Würzburg-Schweinfurt
Fakultät Informatik und Wirtschaftsinformatik

Projektdokumentation

Workflowbasierte Rückbestätigung von Lieferterminen

vorgelegt an der Technischen Hochschule Würzburg-Schweinfurt
in der Fakultät Informatik und Wirtschaftsinformatik
im Rahmen der Projektarbeit

Name 1
Name 2
Name 3

Betreuerin: Frau Prof. Dr. Saueressig

Technischer Betreuer: Herr Prof. Dr. Braun

Abgabedatum: 16.09.2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation und Problemstellung	4
1.2	Zielsetzung	4
1.3	Methodische Vorgehensweise	4
2	IST-Analyse	4
2.1	Bestehender Avisierungsprozess	4
2.2	Bestehende Kommunikationskanäle	4
2.3	Schwachstellen und Optimierungspotenziale	4
2.4	Funktionale Anforderungen	5
2.5	Nichtfunktionale Anforderungen	5
3	SOLL-Konzept	5
3.1	Zielbild des Avisierungsprozesses	5
3.2	SOLL-Prozess	5
3.2.1	Prozessablauf	5
3.2.2	BPMN-Modell	6
4	Systemarchitektur	7
4.1	Gesamtarchitektur	7
4.2	Systemkomponenten	7
4.3	Schnittstellen und Interaktionen	8
5	Backend des Avisierungsservice – Konzeption und Implementierung	10
5.1	Architektur- und Domänenentwurf	10
5.1.1	Architekturprinzipien, Verantwortungsgrenzen und Abhängigkeits- richtung	10
5.1.2	Domänen- und Zustandsmodell	10
5.1.3	Persistenzmodell und Schemaentwicklung	10
5.2	Technische Umsetzung des Avisierungs- und Rückbestätigungsprozesses . .	10
5.2.1	Übernahme von Avisierungsaufträgen, Validierung und Prozessstart	10
5.2.2	Dauerhafte Workflow-Steuerung mit Camunda	10
5.2.3	Kundenreaktion und Fristablauf	10
5.2.4	Erneute Avisierung und Ablösung veralteter Anfragen	10
5.2.5	Statusrückmeldung an DISPO	10
5.2.6	Fehlerbehandlung, Wiederholungen, Nebenläufigkeit und Idempoten- zenzgrenzen	10
5.3	Kanalunabhängige Kundenkommunikation	10
5.3.1	Kommunikationsabstraktion und Nachrichtenerzeugung	10
5.3.2	E-Mail-Versand und mandantenspezifische SMTP-Konfiguration . .	10
5.3.3	SMS und WhatsApp über Twilio: Technologieauswahl und Integration	10
5.4	Mandantenfähigkeit und Sicherheitskonzept	10
5.4.1	Mandantenkontext und Datenisolation	10
5.4.2	Authentifizierung der DISPO-Schnittstelle	10
5.4.3	Zugriffsschutz des Dashboards	10
5.4.4	Tokenbasierter Kundenzugriff	10
5.4.5	Schutz sensibler Konfigurationsdaten	10

5.5	Abfragemodell für die Disposition	10
5.5.1	Aufbau des leseseitigen Abfragemodells	10
5.5.2	Tourübersicht, dynamische Filterung, Gruppierung und Paginierung	10
5.5.3	Auftragsdetails und Avisierungshistorie	10
5.6	Lieferverfolgung und Geocoding	10
5.6.1	Trackingdaten und Fahrerposition	10
5.6.2	Zielkoordinaten und Geocoding	10
5.6.3	Bereitstellung über die Kundenschnittstelle	10
5.7	Technische Konfiguration und Integrationsumgebung	10
5.7.1	Externe Konfiguration und Laufzeitprofile	10
5.7.2	Lokale Infrastruktur und Simulation externer Systeme	10
6	Frontend – Konzeption und Implementierung	10
6.1	Einordnung des Frontends in das Gesamtsystem	10
6.2	Frontend-Architektur und technische Grundlagen	10
6.2.1	Technologieauswahl	10
6.2.2	Seiten-, Komponenten- und Routingstruktur	10
6.2.3	API-Anbindung und Zustandsverwaltung	10
6.2.4	Responsive Design und wiederverwendbare UI-Komponenten	10
6.3	Avisierungsdashboard	10
6.3.1	Geschützter Dashboard-Zugang	10
6.3.2	Tourübersicht und Statusdarstellung	10
6.3.3	Suche, Filterung und Paginierung	10
6.4	Auftragsdetailansicht und erneute Avisierung	10
6.4.1	Auftragsdaten und Avisierungshistorie	10
6.4.2	Erneute Avisierung, Validierung und Fehlerrückmeldung	10
6.5	Bestätigungsseite für Kunden	10
6.5.1	Tokenbasierter Zugriff und Lieferinformationen	10
6.5.2	Bestätigung, Ablehnung und Kundenkommentar	10
6.5.3	Abgeschlossene, abgelaufene und fehlerhafte Anfragen	10
6.6	Live-Tracking	10
6.6.1	Verfügbarkeit und Datenfluss	10
6.6.2	Karten- und Positionsdarstellung	10
6.6.3	Aktualisierung und Fehlerbehandlung	10
7	Test und Validierung	10
7.1	Teststrategie und Testumgebung	10
7.1.1	Testziele, Testebenen und Auswahl der Prüfschwerpunkte	11
7.1.2	Testwerkzeuge und reproduzierbare Testumgebung	11
7.2	Validierung des Avisierungsservice	11
7.2.1	Domänen- und Anwendungslogik	11
7.2.2	REST-Verträge, Eingabevalidierung und Fehlerantworten	11
7.2.3	Authentifizierung, Sitzungsverwaltung und Mandantenisolation	11
7.2.4	Persistenz- und Abfragemodell	11
7.2.5	Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien	11
7.2.6	Ausgehende Schnittstellen und Rückkanäle	11
7.3	End-to-End-Tests	11
7.3.1	Einordnung der Szenariogruppen	11
7.3.2	Testumgebung und technische Struktur	11

7.3.3	Versand und Bereitstellung der Anfrage	12
7.3.4	Kundenzugriff und Anzeige der Lieferdaten	13
7.3.5	Kundenreaktion und DISPO-Rückmeldung	13
7.3.6	Dashboard-Zugang	13
7.3.7	Tourenübersicht, Suche und Filterung	13
7.3.8	Auftragsdetails und erneute Avisierung	14
7.3.9	E-Mail- und SMTP-Einstellungen	14
7.3.10	Lieferverfolgung und Tracking-Verfügbarkeit	15
8	Projektmanagement und Projektreflexion	15
8.1	Projektmanagement	15
8.1.1	Projektphasen	15
8.1.2	Projektplan	15
8.1.3	Projektaufwand	16
8.2	Projektreflexion	16
8.2.1	Herausforderungen	16
8.2.2	Lessons Learned	17
9	Projekttrgebnisse	17
9.1	Gesamtergebnis	17
9.2	Grenzen des Projektergebnisses	17
9.3	Fazit und Ausblick	17
10	Nutzung von Künstlicher Intelligenz	17
11	Verwendete Hilfsmittel	17

1 Einleitung

1.1 Motivation und Problemstellung

Ausgangspunkt des Projekts war die Beobachtung, dass die Rückbestätigung von Lieferterminen in weiten Teilen manuell erfolgte und dadurch mit hohem Abstimmungsaufwand verbunden war. Informationen mussten mehrfach weitergegeben, Rückmeldungen manuell ausgewertet und Statusänderungen separat dokumentiert werden.

Ziel der Projektarbeit war es deshalb, einen strukturierten digitalen Prozess zu entwickeln, der sowohl die fachliche Kommunikation mit dem Kunden als auch die technische Weiterverarbeitung im System unterstützt.

1.2 Zielsetzung

1.3 Methodische Vorgehensweise

2 IST-Analyse

2.1 Bestehender Avisierungsprozess

Im bisherigen Ablauf wurden Liefertermine durch die Disposition vorbereitet und anschließend auf direktem Weg an die Kunden kommuniziert. Rückmeldungen erfolgten je nach Situation per E-Mail, telefonisch oder über informelle Zwischenabsprachen. Die eingehenden Informationen mussten anschließend manuell ausgewertet und an weitere Beteiligte weitergegeben werden.

Eine durchgängige digitale Verarbeitung des Prozesses war dadurch nur eingeschränkt möglich. Insbesondere bei mehreren gleichzeitigen Lieferungen stieg der Koordinationsaufwand deutlich an.

2.2 Bestehende Kommunikationskanäle

2.3 Schwachstellen und Optimierungspotenziale

Die wesentlichen Schwachstellen lagen in der fehlenden Standardisierung der Rückmeldungen und in den Medienbrüchen zwischen Kommunikation, Bearbeitung und Dokumentation. Daraus ergaben sich Verzögerungen, erhöhter Abstimmungsbedarf und ein größeres Risiko für Missverständnisse im weiteren Ablauf.

Zusätzlich fehlte eine einheitliche Statussicht, sodass der aktuelle Bearbeitungsstand eines Liefertermins nicht jederzeit direkt nachvollzogen werden konnte.

2.4 Funktionale Anforderungen

ID	Beschreibung
A1	Das System versendet einen Bestätigungslink über E-Mail, SMS oder WhatsApp.
A2	Der Kunde kann über eine Bestätigungsseite auf den vorgeschlagenen Liefertermin reagieren.
A3	Nach Eingang der Kundenrückmeldung wird der Status automatisch im System aktualisiert.
A4	Bei bestätigten Lieferungen kann am Liefertag eine Fahrerkartenansicht bereitgestellt werden.
A5	Die Disposition erhält über ein Dashboard eine Übersicht aller relevanten Vorgänge und Statuswerte.

2.5 Nichtfunktionale Anforderungen

3 SOLL-Konzept

3.1 Zielbild des Avisierungsprozesses

3.2 SOLL-Prozess

3.2.1 Prozessablauf

Im SOLL-Prozess wird die Kommunikation mit dem Kunden standardisiert ausgelöst und systemseitig begleitet. Nach Eingang eines relevanten Vorgangs wird automatisch eine Benachrichtigung mit einem personalisierten Link versendet. Der Versand kann je nach hinterlegtem Kommunikationsweg über E-Mail, SMS oder WhatsApp erfolgen. Über diesen Link gelangt der Kunde auf eine Bestätigungsseite, auf der er strukturiert auf den vorgeschlagenen Liefertermin reagieren kann.

Diese Rückmeldung wird automatisiert verarbeitet, in einen Status überführt und den beteiligten Stellen ohne manuelle Zwischenschritte bereitgestellt. Dadurch entsteht ein durchgängiger Prozess von der Auslösung bis zur Anzeige des aktuellen Bearbeitungsstands. Erfolgt eine Bestätigung, kann am Tag der Lieferung zusätzlich eine Fahrerkartenansicht bereitgestellt werden, um den Lieferkontext für den Kunden transparent darzustellen. Parallel dazu behält die Disposition über ein Dashboard den Überblick über offene, bestätigte oder problematische Vorgänge.

3.2.2 BPMN-Modell

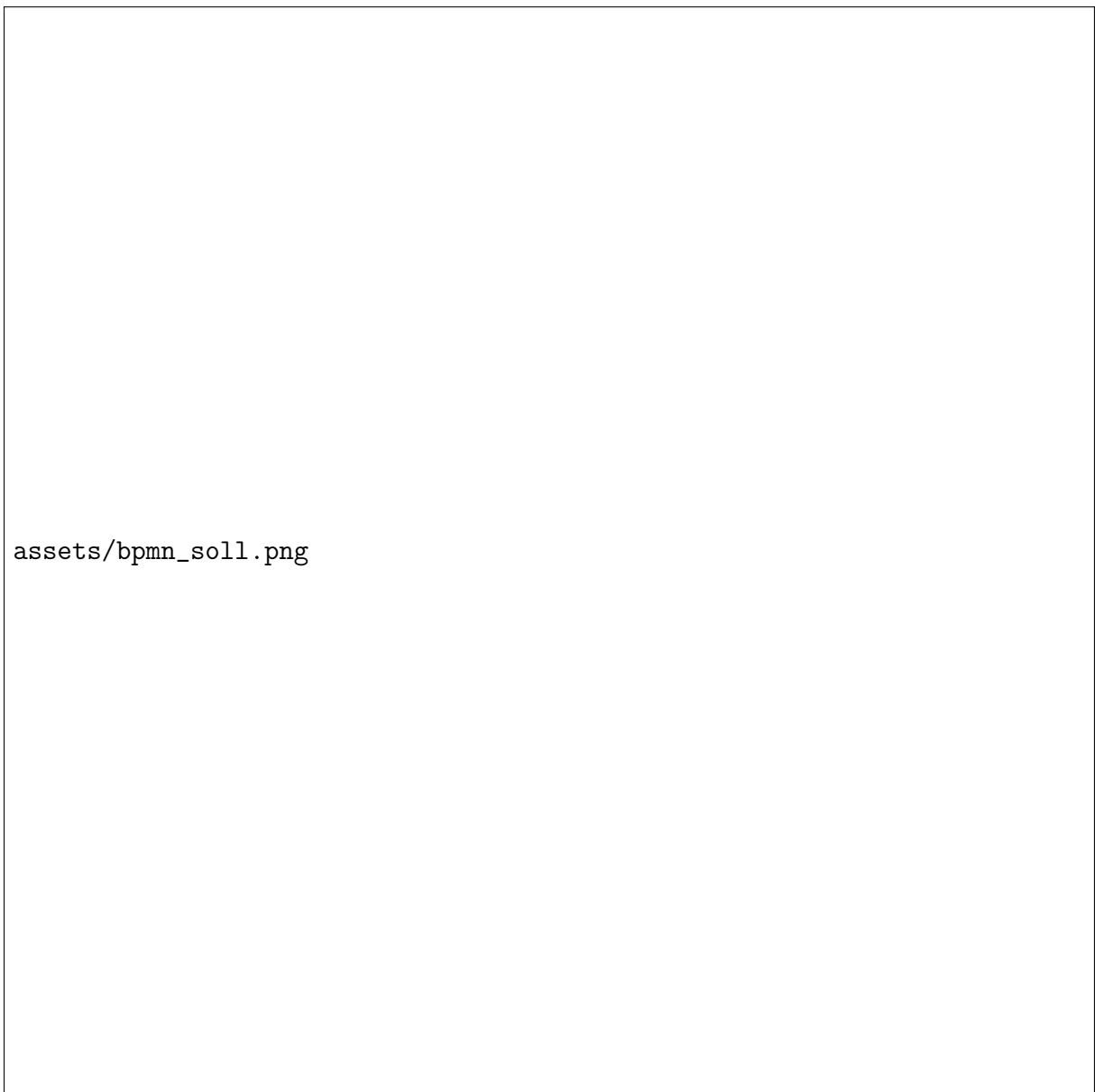


Abbildung 1: BPMN-Modell des SOLL-Prozesses

4 Systemarchitektur

4.1 Gesamtarchitektur



Abbildung 2: Systemarchitektur

4.2 Systemkomponenten

Das Gesamtsystem besteht aus mehreren miteinander kommunizierenden Komponenten. Das Frontend stellt die Benutzeroberflächen für die Disposition und die Kunden bereit. Das Backend übernimmt die fachliche Verarbeitung, stellt Schnittstellen bereit und koordiniert die Kommunikation mit weiteren Systemkomponenten.

Die Workflow Engine unterstützt die Modellierung und Steuerung des Rückbestätigungsprozesses. Persistente Informationen wie Lieferdaten, Rückmeldungen und Statuswerte werden in einer Datenbank gespeichert. Zusätzlich werden externe Dienste für die verschiedenen Kommunikationskanäle angebunden.

4.3 Schnittstellen und Interaktionen

Die Systemkomponenten sind über definierte Schnittstellen miteinander verbunden. Lieferaufträge werden aus dem Dispositionskontext an das Backend übergeben und dort verarbeitet. Das Backend koordiniert die weitere Workflow-Steuerung, persistiert relevante Zustände und stellt die benötigten Daten für die Benutzeroberflächen bereit.

Für die Kommunikation mit dem Kunden werden externe Kommunikationsdienste angebunden. Kundenrückmeldungen werden wiederum vom Backend verarbeitet und stehen anschließend der Disposition über das Dashboard zur Verfügung.

5 Backend des Avisierungsservice – Konzeption und Implementierung

5.1 Architektur- und Domänenentwurf

5.1.1 Architekturprinzipien, Verantwortungsgrenzen und Abhängigkeitsrichtung

5.1.2 Domänen- und Zustandsmodell

5.1.3 Persistenzmodell und Schemaentwicklung

5.2 Technische Umsetzung des Avisierungs- und Rückbestätigungsprozesses

5.2.1 Übernahme von Avisierungsaufträgen, Validierung und Prozessstart

5.2.2 Dauerhafte Workflow-Steuerung mit Camunda

5.2.3 Kundenreaktion und Fristablauf

5.2.4 Erneute Avisierung und Ablösung veralteter Anfragen

5.2.5 Statusrückmeldung an DISPO

5.2.6 Fehlerbehandlung, Wiederholungen, Nebenläufigkeit und Idempotenzgrenzen

5.3 Kanalunabhängige Kundenkommunikation

5.3.1 Kommunikationsabstraktion und Nachrichtenerzeugung

5.3.2 E-Mail-Versand und mandantenspezifische SMTP-Konfiguration

5.3.3 SMS und WhatsApp über Twilio: Technologieauswahl und Integration

5.4 Mandantenfähigkeit und Sicherheitskonzept

5.4.1 Mandantenkontext und Datenisolation

5.4.2 Authentifizierung der DISPO-Schnittstelle

5.4.3 Zugriffsschutz des Dashboards

5.4.4 Tokenbasierter Kundenzugriff

5.4.5 Schutz sensibler Konfigurationsdaten

5.5 Abfragemodell für die Disposition

5.5.1 Aufbau des leseseitigen Abfragemodells

5.5.2 Tourübersicht, dynamische Filterung, Gruppierung und Paginierung

5.5.3 Auftragsdetails und Avisierungshistorie

5.6 Lieferverfolgung und Geocoding

5.6.1 Trackingdaten und Fahrerposition

5.6.2 Zielkoordinaten und Geocoding

5.6.3 Bereitstellung über die Kundenschnittstelle

5.7 Technische Konfiguration und Integrationsumgebung

Frontend, Backend, Workflow Engine und Kommunikationskanälen getestet.

Ein besonderer Fokus lag dabei auf den Übergängen zwischen den Systemkomponenten, da dort fachliche Fehler oder inkonsistente Status besonders wahrscheinlich sind.

7.1.1 Testziele, Testebenen und Auswahl der Prüfschwerpunkte

7.1.2 Testwerkzeuge und reproduzierbare Testumgebung

7.2 Validierung des Avisierungsservice

7.2.1 Domänen- und Anwendungslogik

7.2.2 REST-Verträge, Eingabevalidierung und Fehlerantworten

7.2.3 Authentifizierung, Sitzungsverwaltung und Mandantenisolation

7.2.4 Persistenz- und Abfragemodell

7.2.5 Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien

7.2.6 Ausgehende Schnittstellen und Rückkanäle

7.3 End-to-End-Tests

Die End-to-End-Tests prüfen die zentralen Benutzerabläufe der Kundenbestätigung, des Dispositions-Dashboards und der Lieferverfolgung über Frontend, Backend, E-Mail-Versand und DISPO-Rückmeldung hinweg. Sie werden mit Playwright im Chromium-Browser ausgeführt und befinden sich im Verzeichnis `frontend/tests`.

7.3.1 Einordnung der Szenariogruppen

Die Teststruktur orientiert sich an fachlichen Benutzerabläufen und nicht an einzelnen Frontend-Komponenten. Die Szenarien sind in die Bereiche Kundenbestätigung, Dispositions-Dashboard und Lieferverfolgung gegliedert.

Die Szenarien zur Kundenbestätigung umfassen den Versand und Inhalt der Bestätigungs-E-Mail, den Zugriff auf die Kundenseite, die Anzeige der Lieferdaten sowie die Bestätigung oder Ablehnung eines Liefertermins. Zusätzlich werden ungültige Bestätigungstoken, optionale Kundenkommentare und die Persistenz bereits abgegebener Antworten geprüft. Die Dashboard-Szenarien prüfen den geschützten Zugang, die Tourenübersicht, die Suche und Filterung, die Auftragsdetails, die Avisierungshistorie und die Auswahl des Kommunikationskanals für eine erneute Avisierung. Die Tracking-Szenarien prüfen die Verfügbarkeit und Aktualisierung der Lieferverfolgung.

7.3.2 Testumgebung und technische Struktur

Die Testsuite enthält insgesamt 29 Playwright-Tests. Davon bilden 28 Tests fachliche End-to-End-Szenarien ab. Ein zusätzlicher Setup-Test prüft vor der Ausführung die Erreichbarkeit der benötigten Dienste.

Die fachlichen Tests befinden sich in den Verzeichnissen `frontend/tests/confirmation`, `frontend/tests/dashboard` und `frontend/tests/tracking`.

Die Tests zur Kundenbestätigung sind auf folgende Dateien verteilt:

- `email.spec.ts` für den Versand und Inhalt der Bestätigungs-E-Mail
- `page.spec.ts` für die Anzeige der Kundenseite und das Verhalten bei ungültigen Bestätigungstoken
- `response.spec.ts` für Bestätigung, Ablehnung, Kundenkommentare, DISPO-Rückmeldungen und persistierte Abschlusszustände

Die Tests des Dispositions-Dashboards befinden sich in folgenden Dateien:

- `authentication.spec.ts` für den gültigen und ungültigen Dashboard-Zugang
- `overview.spec.ts` für die Tourenübersicht, Statusanzahlen, Suche, Filterung und Pagination,
- `order-detail.spec.ts` für Auftragsdetails, Anfragehistorie, Kundenreaktionen, Resend-Berechtigung und Kanalauswahl
- `settings.spec.ts` für das Laden und Testen der E-Mail- und SMTP-Einstellungen

Die Tests zur Lieferverfolgung befinden sich in `frontend/tests/tracking/tracking.spec.ts`. Sie prüfen die Tracking-Karte, die Fahrzeug- und Zielmarkierung, die Anzeige der Zieladresse, die Verfügbarkeit vor dem Liefertag und die Aktualisierung der Fahrerposition.

Wiederverwendbare technische Abläufe befinden sich unter `frontend/tests/helpers`. Dazu gehören das Erzeugen eindeutiger Testaufträge, das Auslesen von E-Mails aus Mailpit, das Öffnen einer authentifizierten Dashboard-Sitzung, die Prüfung von DISPO-Rückmeldungen und die Vorbereitung bestätigter Aufträge für Tracking-Szenarien.

Vor den fachlichen Tests führt Playwright den Setup-Test `setup/services.setup.ts` aus. Dieser prüft einmalig die Erreichbarkeit von Backend, Mailpit und DISPO-Mock. Die Prüfungen verwenden Polling, damit die Testausführung auf den Start der benötigten Docker-Dienste warten kann.

Dynamisch erzeugte Testaufträge erhalten mit `randomUUID()` eindeutige Auftragsnummern und E-Mail-Adressen. Die Dashboard-Tests verwenden vorbereitete DEMO-Seed-Daten. Playwright verändert dabei keine Datenbankzustände direkt.

7.3.3 Versand und Bereitstellung der Anfrage

Zur Prüfung des Versandablaufs wird über die geschützte DISPO-Schnittstelle eine Bestätigungsanfrage für einen eindeutigen Testauftrag erzeugt. Die dadurch ausgelöste Bestätigungs-E-Mail wird über den konfigurierten SMTP-Versand an Mailpit übermittelt. Dort wartet der Test auf den Eingang der Nachricht.

Anschließend wird überprüft, ob die E-Mail den Kundennamen, die Lieferadresse, das Produkt, die Bestellmenge, das Lieferzeitfenster, den Preis und den personalisierten Bestätigungslink enthält.

Damit bildet das Szenario den systemübergreifenden Ablauf von der Anlage eines Avisierungsauftrags bis zur Bereitstellung eines für den Kunden nutzbaren Bestätigungslinks ab. Bewertet wird dabei die tatsächlich in Mailpit eingegangene Nachricht.

7.3.4 Kundenzugriff und Anzeige der Lieferdaten

Nach dem Öffnen des Bestätigungslinks wird geprüft, ob die Bestätigungsseite alle für die Kundenentscheidung erforderlichen Informationen anzeigt. Dazu gehören Kundenname, Lieferadresse, Produkt, Bestellmenge, Preis sowie Beginn und Ende des Lieferzeitfensters.

Zusätzlich wird geprüft, wie die Anwendung auf ein unbekanntes Bestätigungstoken reagiert. Sie zeigt in diesem Fall eine verständliche Fehlermeldung an und bietet keine Möglichkeit, den Liefertermin zu bestätigen oder abzulehnen.

7.3.5 Kundenreaktion und DISPO-Rückmeldung

Die Bestätigung und die Ablehnung eines Liefertermins werden als getrennte E2E-Szenarien geprüft.

Nach der Bestätigung muss die Anwendung eine sichtbare Erfolgsmeldung anzeigen. Zusätzlich wird geprüft, ob der DISPO-Mock eine Rückmeldung mit der externen Auftragsnummer und dem Status `CONFIRMED` erhält.

Ein weiterer Test prüft die Übermittlung einer optionalen Kundennachricht im Bestätigungsablauf. Der eingegebene Kommentar muss gemeinsam mit der `CONFIRMED`-Rückmeldung an den DISPO-Mock übertragen werden. Da für Bestätigung und Ablehnung dasselbe Kommentarfeld und derselbe Übertragungsweg verwendet werden, wird die Kommentarübermittlung nur einmal im Bestätigungsablauf geprüft.

Nach der Ablehnung muss die Anwendung ebenfalls eine sichtbare Rückmeldung anzeigen. Der DISPO-Mock muss dabei den Status `REJECTED` für die entsprechende externe Auftragsnummer erhalten.

Die Persistenz beider Abschlusszustände wird durch das erneute Öffnen des Bestätigungslinks geprüft. Der zuvor erreichte Status muss weiterhin angezeigt werden, während die Buttons zum Bestätigen und Ablehnen nicht mehr verfügbar sind. Im abgelehnten Zustand darf außerdem keine Tracking-Karte angezeigt werden.

7.3.6 Dashboard-Zugang

Für den Dashboard-Zugang fordert der Test mithilfe eines gültigen API-Schlüssels über die geschützte Backend-Schnittstelle einen Dashboard-Zugangslink an. Nach dem Öffnen dieses Links im Browser prüft der Test den Aufbau einer authentifizierten Dashboard-Sitzung, die Weiterleitung zum Dispositions-Dashboard und die Anzeige der Tourenübersicht.

Ein weiterer Test ruft die Anwendung mit einem ungültigen Zugangscode auf. In diesem Fall darf das Dashboard nicht angezeigt werden. Stattdessen muss die Anwendung eine verständliche Fehlermeldung ausgeben.

7.3.7 Tourenübersicht, Suche und Filterung

In der Tourenübersicht werden die verfügbaren Touren, die zugehörigen Aufträge und die aggregierten Statusanzahlen kontrolliert. Dabei werden bestätigte, abgelehnte und noch unbeantwortete Aufträge berücksichtigt.

Die globale Suche wird in getrennten Szenarien mit einem Kundennamen, einer Auftragsnummer und einer Lieferadresse ausgeführt. Dabei wird jeweils geprüft, ob der gesuchte Auftrag angezeigt und ein nicht passender Auftrag aus der Ergebnisliste entfernt wird.

Zusätzlich wird die kombinierte Filterung nach Tournummer und Bestätigungsstatus geprüft. Der Pagination-Test wechselt auf die zweite Ergebnisseite, kontrolliert den dafür ausgeführten Backend-Aufruf und prüft die Anzeige der dort enthaltenen Ergebnisse.

7.3.8 Auftragsdetails und erneute Avisierung

Die Auftragsdetailseite zeigt die Stammdaten des Auftrags, den aktuellen Status und die aktuelle Bestätigungsanfrage. Die E2E-Tests prüfen unter anderem die Auftragsnummer, den Kundennamen, die E-Mail-Adresse, die Lieferadresse, das Fahrzeug, die Bestellmenge und den Kommunikationskanal.

Darüber hinaus wird die Avisierungshistorie geprüft. Historische Anfragen mit den Zuständen `REJECTED` und `NO_RESPONSE` sowie gespeicherte Kundenkommentare müssen auf der Detailseite sichtbar sein.

Die Verfügbarkeit der erneuten Avisierung hängt vom aktuellen Status ab. Für einen bereits bestätigten Auftrag muss der Button „Erneut senden“ deaktiviert sein. Für einen Auftrag mit dem Status `NO_RESPONSE` muss der Button dagegen aktiviert sein.

Für eine zulässige erneute Avisierung wird die Auswahl der Kommunikationskanäle E-Mail, SMS und WhatsApp geprüft. Nach der Auswahl eines Kanals muss die vorherige Auswahl aufgehoben und der aktive Kanal über das Attribut `aria-pressed` zugänglich gekennzeichnet werden. Der Button zum erneuten Senden muss dabei aktiviert bleiben.

Der Versand wird im E2E-Test nicht ausgelöst, da die Dashboard-Tests gemeinsam verwendete `DEMO`-Seed-Daten nutzen. Eine erneute Avisierung würde die aktuelle Anfrage und ihre Historie verändern und dadurch die Wiederholbarkeit der Tests beeinträchtigen.

Für SMS und WhatsApp wird ausschließlich die Auswahl in der Benutzeroberfläche geprüft. Ein tatsächlicher Versand über Twilio erfolgt nicht, da er reale Zugangsdaten und Empfänger erfordern sowie eine Abhängigkeit von einem externen Dienst erzeugen würde.

7.3.9 E-Mail- und SMTP-Einstellungen

Die E-Mail-Einstellungen des Dashboards werden in drei Szenarien geprüft.

Zunächst wird kontrolliert, ob die gespeicherten SMTP-Einstellungen vom Backend geladen und in der Benutzeroberfläche dargestellt werden.

Anschließend wird über den Button „Verbindung testen“ geprüft, ob das Backend die gespeicherte SMTP-Konfiguration erfolgreich verwenden kann. Nach erfolgreichem Abschluss muss eine entsprechende Meldung angezeigt werden.

Der dritte Test versendet eine SMTP-Test-E-Mail. Neben der sichtbaren Erfolgsmeldung wird geprüft, ob Mailpit tatsächlich eine neue Nachricht mit dem erwarteten Betreff erhalten hat.

7.3.10 Lieferverfolgung und Tracking-Verfügbarkeit

Für bestätigte Lieferungen am Liefertag wird geprüft, ob die Live-Tracking-Ansicht verfügbar ist. Die Tracking-Schnittstellen müssen gültige Ziel- und Fahrerkoordinaten liefern. In der Benutzeroberfläche müssen die Tracking-Karte sowie die Fahrzeug- und Zielmarkierung sichtbar sein.

Ein weiteres Szenario prüft, ob die Lieferadresse als Zieladresse angezeigt wird und die Zielmarkierung auf der Karte sichtbar ist.

Bei einer bestätigten Lieferung mit einem zukünftigen Lieferdatum meldet die Tracking-Schnittstelle, dass die Lieferverfolgung noch nicht verfügbar ist, und liefert keine Zielkoordinaten. Die Anwendung zeigt daher weder die Tracking-Karte noch die Fahrzeug- und Zielmarkierung an. Stattdessen wird der Kunde darauf hingewiesen, dass die Tracking-Informationen erst am Liefertag verfügbar sind.

Abschließend wird die Aktualisierung der Fahrerposition über den Button „Aktualisieren“ geprüft. Dabei müssen neue Fahrerkoordinaten geladen und der angezeigte Lieferstatus beziehungsweise die verbleibende Entfernung aktualisiert werden. Die Karte sowie die Fahrzeug- und Zielmarkierung müssen weiterhin sichtbar bleiben.

8 Projektmanagement und Projektreflexion

8.1 Projektmanagement

8.1.1 Projektphasen

Das Projekt wurde in klar abgegrenzte Phasen unterteilt, um Analyse, Konzeption, Umsetzung und Validierung strukturiert durchführen zu können. Die Übergänge zwischen den Phasen waren teilweise iterativ, da Erkenntnisse aus der Implementierung wieder in die Ausgestaltung des SOLL-Prozesses eingeflossen sind.

Phase	Zeitraum	Status
Analyse und Anforderungsabstimmung	xx.xx - xx.xx	abgeschlossen
Architektur und Prozessmodellierung	xx.xx - xx.xx	abgeschlossen
Implementierung	xx.xx - xx.xx	abgeschlossen
Integration und Test	xx.xx - xx.xx	abgeschlossen
Abschluss und Ergebnisaufbereitung	xx.xx - xx.xx	abgeschlossen

8.1.2 Projektplan

Der Projektplan diene als gemeinsame Orientierung für inhaltliche Meilensteine, Abstimmungstermine und die Reihenfolge der Umsetzung. Besonders hilfreich war die frühe Sichtbarkeit von Abhängigkeiten zwischen Prozessmodell, Schnittstellen und Benutzeroberfläche.

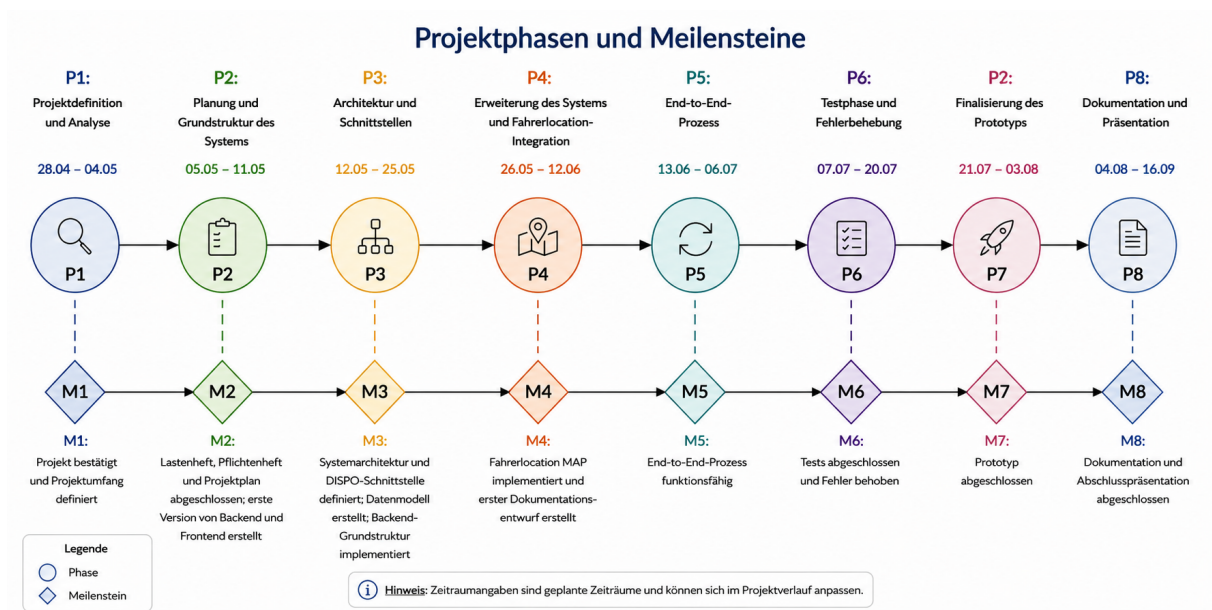


Abbildung 3: Projektplan

8.1.3 Projektaufwand

Der dokumentierte Aufwand verteilt sich auf Analyse, Konzeption, Implementierung, Test und Abstimmung mit dem Auftraggeber. Die Tabelle dient als Beispielstruktur zur Erfassung der geleisteten Stunden und kann mit den tatsächlichen Projektdaten vervollständigt werden.

Datum	Person	Tätigkeit	Stunden
xx.xx.2026	Name 1	Anforderungsanalyse und Recherche	4
xx.xx.2026	Name 2	Backend-Implementierung	6
xx.xx.2026	Name 3	BPMN-Modellierung und Camunda-Integration	5
xx.xx.2026	Name 1	Frontend-Umsetzung und UI-Anpassungen	5
xx.xx.2026	Team	Integrationstest und Ergebnisabstimmung	4

8.2 Projektreflexion

8.2.1 Herausforderungen

Herausfordernd war insbesondere die Abstimmung an den Schnittstellen zwischen fachlicher Prozessbeschreibung und technischer Umsetzung. Bereits kleine Unklarheiten bei Statusübergängen oder Rückmeldewegen hatten direkte Auswirkungen auf mehrere Systemkomponenten.

Auch die Konsistenz zwischen Benutzerführung, Datenmodell und Workflowlogik erforderte wiederholte Abstimmungen.

8.2.2 Lessons Learned

Für vergleichbare Projekte empfiehlt es sich, Statusmodelle, Ereignisse und Verantwortlichkeiten frühzeitig gemeinsam zu definieren und schriftlich festzuhalten. Darüber hinaus hat sich gezeigt, dass ein visuelles Prozessmodell in Kombination mit einer frühen technischen Teilimplementierung sehr hilfreich ist, um Missverständnisse schnell sichtbar zu machen.

9 Projektrgebnisse

9.1 Gesamtergebnis

Die Projektarbeit führte zu einem konsistenten Prototypen, der die wesentlichen Anforderungen des definierten SOLL-Prozesses abbildet.

- Funktionsfähiger Prototyp
- Frontend-Backend-Kommunikation
- Automatisierte Kommunikation über verschiedene Kanäle
- Digitale Rückbestätigung von Lieferterminen
- Statusverwaltung
- Workflow-Steuerung
- Avisierungsdashboard für die Disposition
- Bestätigungsseite für Kunden
- Lieferverfolgung bzw. Fahrerkartenansicht

9.2 Grenzen des Projektergebnisses

9.3 Fazit und Ausblick

Das Projekt konnte sowohl aus technischer als auch aus organisatorischer Sicht erfolgreich abgeschlossen werden. Neben dem entwickelten Prototypen entstand eine belastbare Grundlage für die weitere fachliche und technische Ausarbeitung des Systems.

Die Kombination aus Prozessmodellierung, Webanwendung und automatisierter Kommunikation erwies sich als geeigneter Ansatz für die Digitalisierung des betrachteten Anwendungsfalls.

10 Nutzung von Künstlicher Intelligenz

11 Verwendete Hilfsmittel